

ChatImage: Interactive Visual Images for Long-Form LLM Answers

ChatImage Contributors

Open-source project

Large Language Models can answer complex questions, but their answers often arrive as dense linear text that is useful but difficult to navigate. CHATIMAGE turns that answer into an *interactive visual image*. The system starts with a normal long-form answer, normalizes it into visual modules, plans an image, and renders the picture. A second pass analyzes the rendered image with vision grounding models such as LocateAnything and MiMo-Vision, optionally refines regions with SAM-style masks, and places transparent clickable hotspots on the content visible in the rendered image. Each hotspot opens a detail panel and a follow-up thread scoped to that region, so the user can inspect one part of the answer without rereading the full response. CHATIMAGE treats planned coordinates as priors rather than final interaction geometry: it generates first, then grounds the visible result. We provide an open-source reference system, a 30-question benchmark across infographic, map, and scene modes, and a real-provider evaluation with completion checks, a strict visual-alignment gate, and a SAM mask-completeness diagnostic. Code, demos, and a live project page are publicly available.

 **Date:** July 2, 2026

 **Correspondence:** <https://github.com/wencanjiang/ChatImage/issues>

 **Code:** <https://github.com/wencanjiang/ChatImage>

 **Project:** <https://wencanjiang.github.io/ChatImage/>



1 Introduction

Large Language Models (LLMs) have become a primary interface for answering complex questions [1, 25]. The interaction format, however, remains largely linear: the model writes a long answer, and the user scrolls through it. This format works for short factual responses, but it becomes inefficient when the answer has internal structure. When a user asks "compare REST and GraphQL" or "explain the RAG retrieval loop," they often want to inspect one module, one tradeoff, or one stage. Plain text can contain that information, but it provides little support for navigation.

Parallel advances in text-to-image generation [26, 27, 30] and vision-language understanding [4, 17] suggest a better interface: *render the answer as an image*. A generated infographic, map, or scene can show modules, hierarchy, routes, and spatial relationships at a glance. Static images, however, discard much of the explanatory structure behind each region and provide no local follow-up mechanism. Directly overlaying planned boxes is also fragile because image generators do not reliably place content at the requested coordinates [6, 8]. If the interaction layer is copied from the pre-generation plan, the user may click an empty area or the wrong object.

We propose CHATIMAGE, a system that turns a long-form LLM answer into an **interactive visual image**: a generated picture with transparent hotspots over the visible regions (Figure 1). Each hotspot opens the explanation for that region and carries its own follow-up thread. The central engineering problem is alignment: a hotspot should cover the object, panel, landmark, or diagram region the user can actually see, not merely the box that was planned before generation.

CHATIMAGE handles this with a **generate-then-ground interaction loop**. The system turns the answer into a small set of visual modules, plans an approximate layout, writes an image prompt, and generates the image. A second pass sends the rendered image through visual grounding, finds where each module actually appeared, and places clickable hotspots on those discovered regions. This decouples *what the answer should*

ChatImage: turning dense language model answers into navigable visual interaction

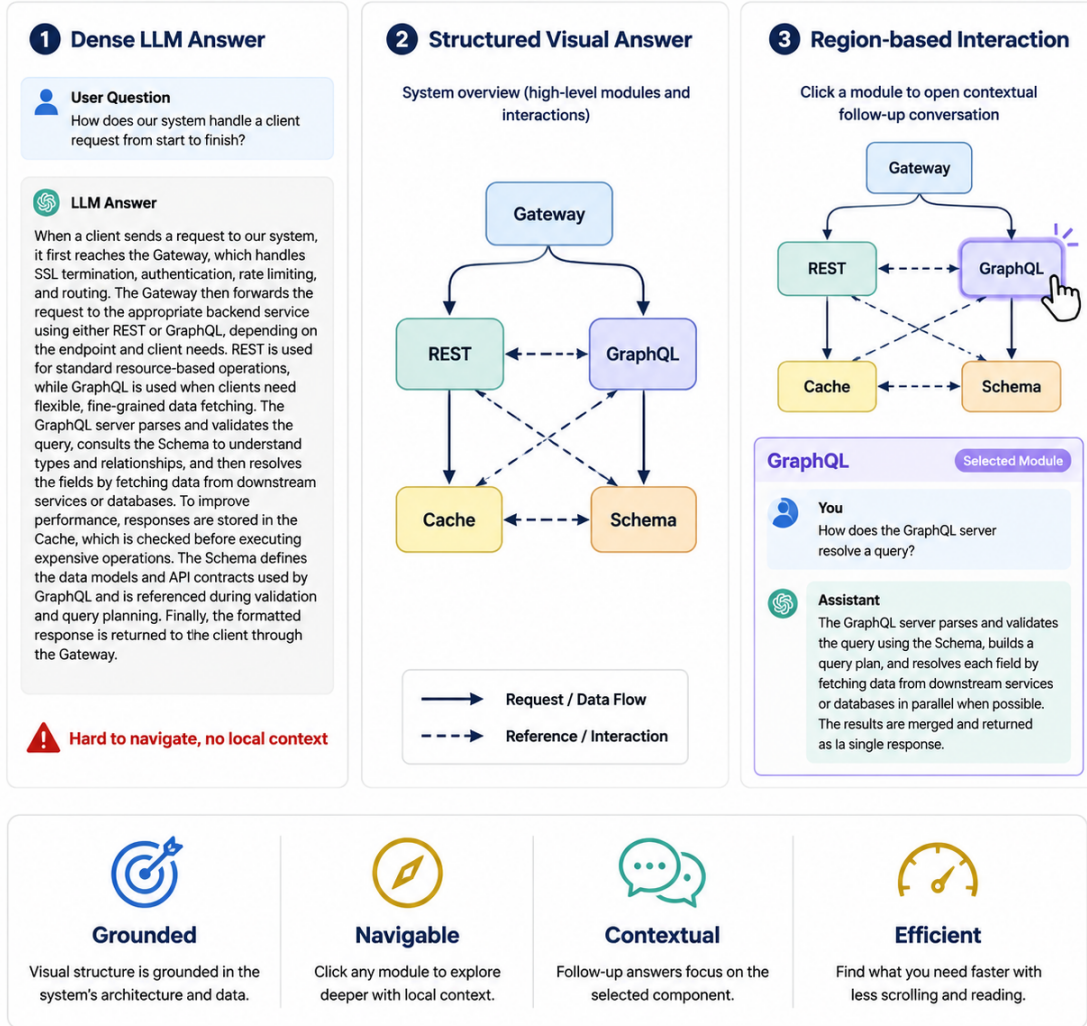


Figure 1 CHATIMAGE turns a long-form LLM answer into an interactive visual image. Given a user question, CHATIMAGE generates an image whose regions are clickable hotspots. Each hotspot opens a detail panel with the region's explanation and an in-context follow-up thread, so the user explores only what they need.

contain from where the image generator rendered it. The resulting artifact supports region-level exploration: the user clicks a visible region, sees the relevant detail and a local preview, and asks a follow-up without re-explaining the entire answer. The system is provider-agnostic: it runs in a no-key mock mode for development and demos, or proxies real text, image, and vision models through a local backend so API keys never reach the browser.

To our knowledge, prior work has not treated LLM answers as navigable, region-interactive visual images with vision-grounded hotspots. Our contributions are:

- **Interactive visual answers.** We introduce the task of rendering a long-form LLM answer as an explorable image with clickable hotspots, detail panels, and per-region follow-up threads.
- **Generate-then-ground alignment.** We decouple structured content from visual placement: the system first renders the image, then uses visual grounding and optional SAM-family mask refinement to place hotspots on actual visible regions rather than a hard-coded grid.

- **Open-source reference system.** We release a complete, provider-agnostic implementation with deterministic mock mode, real API mode, local SQLite persistence, per-hotspot follow-up threads, and a calibration mode for inspecting and correcting hotspot bounds.
- **Benchmark and grounding evaluation.** We assemble a 30-question benchmark spanning multiple content types and evaluate the real-provider interaction loop with automatic completion checks and a strict visual-alignment gate.

2 Related Work

CHATIMAGE sits at the intersection of text-to-image generation, visual grounding, interactive visualization, and LLM answer presentation. As the task of turning an LLM answer into a region-interactive visual image is, to our knowledge, new, we review the closest adjacent lines and the gap CHATIMAGE fills.

Text-to-image and controllable generation. Diffusion models [26, 27, 29, 30] produce high-fidelity images but do not follow spatial instructions precisely: "place module A top-left" yields no guarantee [6, 8]. Controllable methods such as ControlNet [37], GLIGEN [16], training-free layout-to-image [3], and InfographicNLP [19] impose structure but target synthesis quality, not making generated regions *interactive* and *aligned with a structured answer*. CHATIMAGE uses text-to-image as a rendering backend but does not rely on its layout fidelity: it discovers region locations *after* generation via grounding.

Visual grounding and segmentation. Referring localization methods such as MDETR [13], TransVG [7], UNINEXT [35], Grounding DINO [18], and YOLO-World [5], as well as grounding-oriented VLMs [4, 17, 23, 34] output boxes for descriptive prompts; SAM/SAM2 [14, 28] refine a box into a mask. CHATIMAGE repurposes this family for a new goal: not finding a pre-existing object, but *aligning a structured answer's regions to their actual rendered positions* in a freshly generated image.

Interactive visualization, VQA, and structured LLM output. Interactive image annotation [11, 22] and VQA [2, 10, 20, 21] explore *fixed* images rather than answer-generated ones, and VQA treats the image as input to be queried, not as an *answer* to be navigated. Structured-output work such as chain-of-thought [33], ReAct [36], tool use [31], and RAG [9, 15] structures *reasoning*, not the output *modality*; the answer stays text. Mind-map and chart-generation methods [12, 24, 32, 38] visualize structure but as static outputs without image-level region interaction or grounding. CHATIMAGE combines these threads into an underexplored task: an LLM answer rendered as a navigable, region-interactive visual image with vision-grounded hotspots, for which we provide an open-source system, benchmark, and evaluation.

3 Method

CHATIMAGE converts a user's question into an interactive visual image by separating two jobs that are easy to conflate. The system decides *what* the answer should contain, but it waits until after image generation to discover *where* that content actually appeared. This section describes the end-to-end loop (Section 3.1), the structured answer normalization and layout planning (Section 3.2), the two-pass vision alignment that grounds hotspots to real image content (Section 3.3), and the interactive hotspot layer and per-region follow-up threads (Section 3.4).

3.1 End-to-End Interaction Loop

Given a user question q , CHATIMAGE produces an interactive visual image $\mathcal{I}$ with a set of clickable hotspots $\mathcal{H}$. The loop proceeds in two passes. The first pass plans what should be shown and generates the image; the second pass checks where the generated content actually appears and derives the clickable hotspot layer from that evidence.

Pass 1: Structure, layout, and image generation.

1. **Raw answer.** Obtain a long-form LLM answer a to q via a text model (or a deterministic mock for development).
2. **Normalization.** Normalize a into a structured visual specification $\mathcal{S}$, a set of modules $\{m_i\}$ and auxiliary modules, each with a title, detail text, region prompt, and metadata (Section 3.2).
3. **Layout planning.** Plan a layout $\mathcal{L}$ that assigns each module a region r_i with normalized bounds in $[0, 1]^4$.
4. **Image prompt synthesis.** Derive an image prompt from $\mathcal{S}$ and $\mathcal{L}$, encoding the visual style and the intended region labels and positions.
5. **Image generation.** Generate the image via a text-to-image model, obtaining $\mathcal{I}$ and its real pixel dimensions (W, H) parsed from the image header (not the request, since generators do not guarantee exact sizes).

Pass 2: Vision alignment and hotspot derivation.

6. **Visual grounding.** Send $\mathcal{I}$, (W, H) , and the modules with their planned bounds to a visual grounding model, which returns a grounded box b_i for each module where it was actually rendered (Section 3.3).
7. **Mask refinement (optional).** Optionally refine each b_i with the SAM3 adapter to obtain a pixel mask for tighter previews.
8. **Hotspot derivation.** Copy the grounded bounds into rectangular hotspot geometry and render transparent, clickable hotspots over $\mathcal{I}$ (Section 3.4).

CHATIMAGE **does not assume the image generator follows the planned layout**. The loop treats layout planning as a prior and visual grounding as the source of final hotspot geometry. This design avoids requiring precise spatial control from diffusion models, which current generators do not reliably provide [8], and ties clickable regions to rendered content instead of planned coordinates.

3.2 Structured Answer Normalization and Layout

Visual specification. The raw answer a is parsed into a visual specification $\mathcal{S}$ with a visual mode (infographic, map, scene, or poster), a summary, and a list of modules. Each module m_i carries: *title* (the hotspot label), *detail* (the explanation shown in the detail panel), *imageText* (short on-image caption), *regionPrompt* (a locator phrase for visual grounding), and metadata (*regionKind*, *iconHint*, *maskPolicy*, locator queries). Because LLMs sometimes echo prompt scaffolding into user-facing text, the system runs a sanitization pass before anything is shown: visual-system vocabulary, meta-instructions, and obvious prompt fragments are removed, with short labels used as a fallback when a detail becomes unusable. When the LLM returns no valid structure, CHATIMAGE falls back to deterministic mock specifications for common infographic, map, and scene topics, ensuring the app still produces a valid visual answer in no-key development mode.

Layout planning. A layout planner assigns each module a region r_i with normalized bounds $\in [0, 1]^4$ relative to the image canvas. Layouts are organized by family, including grid, flow (swimlane), compare, hub, timeline, matrix, and freeform layouts, selected by the visual mode and relation type. For maps, a semantic-slot catalog assigns regions by keyword matching (e.g., "entrance" maps to bottom-center, "core area" maps to center); for infographics, index-based templates place modules in flow or grid order. These *planned* bounds serve as priors for the grounding step and as a fallback when grounding fails, but they are *not* assumed to match the generated image exactly.

3.3 Two-Pass Vision Alignment

The core technical contribution is aligning the structured regions to the generated image. CHATIMAGE sends the image, its real dimensions, the modules, and their planned bounds to a vision backend that runs a grounding-and-repair loop:

Grounding provider. The grounding step is provider-agnostic: any vision backend that can return a normalized bounding box for a locator phrase can be used. Our reference system supports a dedicated referring-localization model (**LocateAnything-3B** [23]) and a general vision-language model used in structured grounding mode (**MiMo-Vision** [34]). For each module the backend receives the generated image plus locator phrases built from the module title, region prompt, and semantic hints. It returns candidate boxes in normalized coordinates. The system compares those candidates with the planned layout, keeps the most plausible one, and can re-ground inside a crop when the first box is too uncertain. This is a *referring localization* step: the system recovers the rendered location of an answer module rather than detecting a predefined object class.

Fallbacks. When the grounding backend rejects a module or is unavailable, CHATIMAGE falls back along a graded chain: a general VLM-based grounding pass, then local OCR, and finally the planned bounds. Each module is tagged with its alignment source (e.g., *locateanything*, *mimo-vision*, *sam3-refined-planned*, *planned-fallback*, *vision-low-confidence*) so the system can report alignment quality per region rather than as a single aggregate score.

Robustness to partial failure. A single missing or low-confidence grounding must not corrupt the whole result. The system rejects groundings that look like header strips or cross-panel artifacts (falling back to planned bounds for those) while keeping far-from-planned groundings that still localize the correct region (tagged low-confidence). Modules entirely absent from the grounding response retain their planned bounds and are flagged, rather than discarding the alignments of all correctly-grounded modules. Each grounded box is repaired for clickability by enforcing a minimum area and safe margin before becoming a hotspot.

SAM3 mask refinement (optional). For tighter visual previews, an interactive box-prompted segmentation model in the SAM family [14, 28] takes each grounded box as a prompt and predicts a pixel mask, from which a tightened box and polygon are derived. The mask is used for preview rendering, including cutouts, organic feathering, and soft fades, while click hit-testing remains rectangular in the normal interaction loop. This keeps the interaction dependable even when a mask is visually useful but not clean enough to serve as the clickable shape.

3.4 Interactive Hotspot Layer and Follow-up Threads

Hotspot rendering. Each hotspot is a transparent, absolutely-positioned button over the image at its grounded coordinates. In normal use, hotspots are fully invisible so the generated image is the only thing the user sees; a hover state provides a soft visual hint without altering the image. A calibration mode toggles visible outlines and per-hotspot labels for inspecting and correcting bounds.

Detail panels and previews. Clicking a hotspot opens a centered detail panel showing the module’s title, sanitized detail text, a localized preview of the image region, and a follow-up input. The preview variant is chosen by a strategy module based on the hotspot shape and mask availability: cutout (checkerboard background), organic feathering, soft radial fade, or mask-based polygon crop.

Per-region follow-up threads. Each hotspot owns an isolated follow-up thread. A follow-up question is sent to the text model with a *local* context: the original question, the raw answer, the clicked hotspot’s detail, and that thread’s history. The answer is scoped to the region without re-explaining the whole image. Threads are persisted alongside the ChatImage, enabling resumption across sessions.

3.5 Implementation

CHATIMAGE is a provider-agnostic web system. The browser frontend is vanilla JavaScript; the local backend runs on Node.js, proxies text/image/vision providers, and stores generated ChatImages, hotspot metadata, alignment traces, and follow-up threads in SQLite. Vision models run behind long-lived local adapters so the browser never sees API keys, checkpoint paths, or raw provider credentials. The same app supports a

no-key *mock* mode with deterministic local providers, and an *api* mode that calls configured real providers. The code and demos are open source, with a live project page for inspection and reproduction.¹

4 Experiments

We evaluate CHATIMAGE along two axes. First, we measure basic completion: whether a benchmark prompt yields a complete ChatImage, consisting of a generated picture and its hotspot layer. Second, we measure hotspot evidence: whether a clickable region is backed by a visual grounding source and usable mask-preview data. The reported completion rate is computed over valid real-provider runs; attempts stopped by upstream API quota limits are treated as external run-limit events rather than interaction-loop failures.

4.1 Benchmark

Table 1 Benchmark composition and scenario coverage. The 30-question benchmark covers structured analytical answers (technical/business layouts), spatial guidance tasks, and object-rich everyday scenes.

Mode	#Q	Avg. hotspots	Scenario coverage	Alignment stress
Infographic	15	5.0	technical workflows, comparisons, decision aids, checklists, timelines	compact panels, dense labels, semantically similar regions
Map	5	6.0	tour routes, landmark clusters, spatial itineraries, scenic areas	route-like geometry, landmark placement, irregular spatial regions
Scene	10	6.0	food options, coffee shops, reading spaces, record stores, plant care	object localization, overlapping items, natural scene composition
Total	30	5.5	three visual modes	generation plus hotspot grounding

The benchmark contains 30 questions designed to exercise different visual modes and content types, summarized in Table 1. The current public showcase is a curated subset of the same workflow after strict visual-alignment screening, including selected map and everyday-scene demos. Map questions stress geographic placement and route-like scenic areas; scene questions stress object localization in less rigidly structured images; infographic questions stress compact analytical panels and decision-support layouts. Each benchmark question is run through the full CHATIMAGE interaction loop with real text, image, and vision providers. Only outputs that pass the stricter public-demo gate are published on the demo page.

Because the grounding step is provider-agnostic (Section 3.3), different regions can be aligned by different signals: a dedicated localizer, a VLM grounding fallback, OCR, a SAM-refined planned box, or a planned fallback. The evaluation reports both completion and the source of each logged hotspot alignment. Representative qualitative results are shown in Figure 2.

4.2 Interaction-Loop Results

Table 2 Hotspot quality diagnostic. Rates use the same 24-hotspot checked subset. "Strict-gate pass" counts hotspots accepted by the automatic grounding and mask-quality gate used before demo promotion. "SAM-complete" counts masks without holes or empty cavities. The complementary rows expose how much of the checked subset still needs better grounding or segmentation.

Metric	Unit	Count	Rate
Strict-gate passes ↑	Hotspot	17/24	70.8%
Strict-gate rejects ↓	Hotspot	7/24	29.2%
SAM-complete masks ↑	Hotspot	13/24	54.2%
SAM-incomplete masks ↓	Hotspot	11/24	45.8%

Figure 3 gives a compact visual summary of hotspot quality, alignment-source mix, and the SAM mask-completeness diagnostic.

Across valid real-provider runs that reached the generation step, all 30 benchmark questions produced a complete ChatImage with a generated image and hotspot layer, yielding a basic completion rate of 100.0%.

¹<https://github.com/wencanjiang/ChatImage>

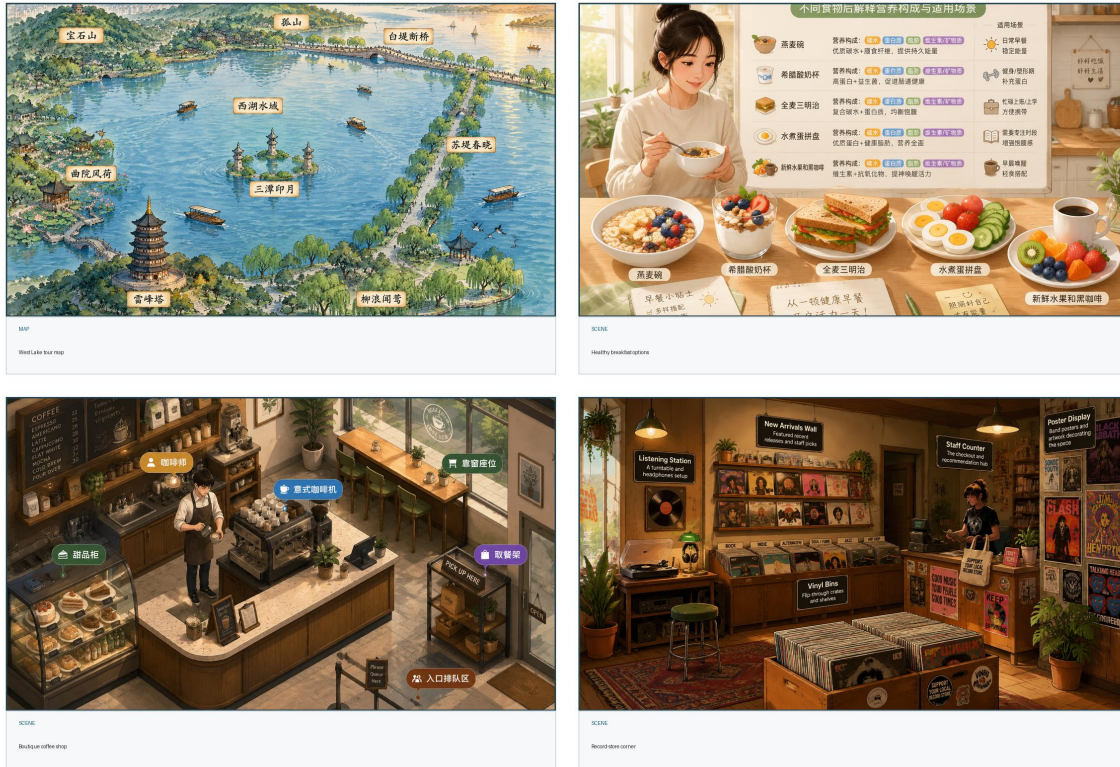


Figure 2 Qualitative results. Example CHATIMAGE outputs across map, infographic, and scene modes. Each generated image is paired with a structured hotspot layer whose regions open detail panels in the interactive demo.

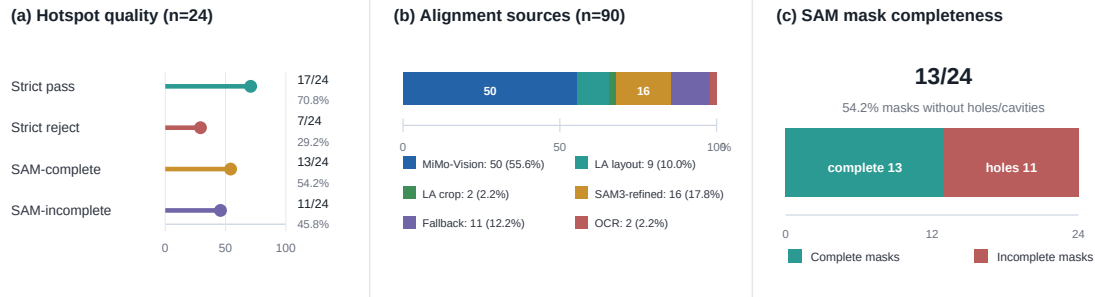


Figure 3 Experimental metric summary. Hotspot-quality diagnostics, alignment-source distribution, and the SAM mask-completeness check; SAM-complete requires masks without obvious holes or empty cavities. Attempts stopped by upstream quota limits are excluded from this denominator and treated as external run-limit events. This is a sanity check for valid API runs rather than the primary diagnostic result. The more informative measurement is hotspot reliability: Table 2 uses a shared 24-hotspot checked subset to report both accepted and rejected cases under the strict visual-alignment gate. The gate accepted **17 of 24 checked hotspots** (70.8%) and rejected 7 (29.2%). This gate is intentionally conservative: each accepted hotspot needs a primary visual grounding signal and usable preview geometry or mask data, so weak or missing visual evidence is rejected rather than promoted. The SAM-complete row is a stricter mask-shape diagnostic within the same 24 checked hotspots: **13** pass the SAM segmentation completeness check (54.2%), while 11 remain incomplete (45.8%), meaning the mask has obvious holes or empty cavities.

4.3 Alignment Source Analysis

Table 3 Alignment source distribution. Source assigned to the 90 logged hotspot alignments from the diagnostic source sample. Primary visual grounding combines MiMo-Vision and LocateAnything; planned-derived rows are retained by the robustness fallback when primary grounding is missing or rejected.

Alignment source	Hotspots	Share
MiMo-Vision primary grounding	50	55.6%
LocateAnything layout-guided grounding	9	10.0%
LocateAnything crop grounding	2	2.2%
SAM3-refined planned region	16	17.8%
Planned fallback	11	12.2%
Local OCR support	2	2.2%
Total	90	--

Table 3 reports the grounding source distribution for 90 logged hotspot alignments from the diagnostic source sample. Most logged hotspots are supplied by primary visual grounding: MiMo-Vision accounts for 50 hotspots (55.6%), while LocateAnything contributes another 11 hotspots when combining layout-guided and crop-based calls (12.2%). SAM3-refined planned regions account for 16 hotspots (17.8%), and planned fallback is used for 11 hotspots (12.2%). The source distribution shows that the system can distinguish visually grounded regions from fallback regions: it records which signal placed each hotspot, keeps high-confidence visual groundings when available, refines plausible boxes with masks, and falls back to planned bounds only when stronger evidence is unavailable.

5 Conclusion

We introduced CHATIMAGE, a system that turns a long-form LLM answer into an interactive visual image: a generated picture overlaid with clickable hotspots, where each region opens its own explanation and follow-up thread. The main design choice is practical: because the image generator is not guaranteed to obey a layout, the interaction layer should not be copied directly from the pre-generation plan. CHATIMAGE generates the image first and grounds the visible result afterward. The open-source implementation spans infographic, map, and scene modes, and the evaluation reports completion, hotspot source attribution, and a strict visual-alignment gate.

Task positioning. CHATIMAGE frames interactive visual answers as a systems problem rather than a model-building problem. The contribution is a system pattern: structure the answer, render it, ground the rendered regions, and attach local interaction to each region.

Use cases. The interactive visual answer format applies naturally to domains where long answers already contain visual or spatial structure. In **education**, a student can explore a complex process (e.g., the HTTP request-rendering loop) by clicking the stage they care about rather than reading an entire explanation. In **technical documentation and knowledge management**, answers become navigable artifacts where each region supports targeted follow-up, reducing re-reading. In **tourism and spatial guidance**, hand-drawn maps with explorable landmarks and routes offer a more intuitive interface than text directions. In **data communication**, infographic answers make comparisons and funnels scannable while preserving drill-down detail. Better text-to-image and visual-grounding models should improve the fidelity and alignment of these artifacts.

Limitations and future work. The current system is limited by the vision models it relies on. Dense infographics, small objects, and complex maps can still produce weak or partial groundings. Click targets also remain rectangular in the normal interaction loop. Polygon-level hit testing would better fit irregular regions when masks provide a cleaner preview. File context is still text-oriented, so richer document parsing remains future work. Useful next steps include shareable HTML exports, stronger OCR-aware grounding, mask-based interaction, automated visual QA for hotspot accuracy, and synchronized cross-device history.

The open-source release and benchmark are intended to make this format easier to inspect, reproduce, and extend.

Open source. Code, demos, a live project page, and a technical report are publicly available.²

References

- [1] Anthropic. The Claude model family. *Anthropic Technical Report*, 2024. 1
- [2] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. VQA: Visual question answering. In *IEEE/CVF International Conference on Computer Vision*, 2015. 3
- [3] Weixuan Chen, Bohan Liu, Daoan Zeng, Shitian Li, Liangli Huang, Chang Xu, and Chunjie Huang. Training-free layout-to-image generation with latent diffusion models. *arXiv preprint arXiv:2401.13391*, 2024. 3
- [4] Zhe Chen, Jiannan Wu, Wenhai Wang, Weijie Su, Guo Chen, Sen Xing, Muyan Zhong, Qinglong Zhang, Xizhou Zhu, Lewei Lu, et al. InternVL: Scaling up vision foundation models and aligning for generic visual-linguistic tasks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 1, 3
- [5] Tianheng Cheng, Liming Song, Yixiao Ge, Wenhao Liu, Xinggang Wang, and Ying Shan. YOLO-World: Real-time open-vocabulary object detection. *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024. 3
- [6] Yifu Cheng, Peng Zhou, Jiafei Cai, Zhongjie Wei, Wei Sun, Robby T Tan, and Gang Hua. Intelligent IG: Instruction-driven visual infographic generation and editing. *arXiv preprint arXiv:2312.14390*, 2024. 1, 3
- [7] Zhengyuan Deng, Jun Wang, Zihang Zhang, Yufan He, and Jianhua Feng. TransVG: Visual grounding with transformers. In *IEEE/CVF International Conference on Computer Vision*, 2021. 3
- [8] Weixi Feng, Wanrong Zhu, Tsu-Jui Fu, Varun Jampani, Arjun Akula, Xuehai He, Sugato Basu, Xin Eric Wang, and Michael S Bernstein. LayoutGPT: Compositional visual planning and generation with large language models. In *Advances in Neural Information Processing Systems*, 2023. 1, 3, 4
- [9] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2024. 3
- [10] Yash Goyal, Tejas Khot, Douglas Summers-Stay, Dhruv Batra, and Devi Parikh. Making the V in VQA matter: Elevating the role of image understanding in Visual Question Answering. *arXiv preprint arXiv:1612.00837*, 2017. 3
- [11] Harry Hochheiser and Ben Shneiderman. Interactive poster visualization with PhD Online. *IEEE Computer Graphics and Applications*, 2004. 3
- [12] Yiyang Hu, Liang Pang, Yanyan Lan, Jiafeng Xu, and Xueqi Cheng. Can large language models generate visual mind maps from text? *arXiv preprint arXiv:2402.10988*, 2024. 3
- [13] Aishwarya Kamath, Mannat Singh, Yann LeCun, Gabriel Synnaeve, Ishan Misra, and Nicolas Caron. MDETR: Modulated detection for end-to-end multi-modal understanding. In *IEEE/CVF International Conference on Computer Vision*, 2021. 3
- [14] Alexander Kirillov, Eric Mintun, Nikhila Ravi, Hanzi Mao, Chloe Rolland, Laura Gustafson, Tete Xiao, Spencer Whitehead, Alexander C. Berg, Wan-Yen Lo, Piotr Dollár, and Ross Girshick. Segment anything. In *IEEE/CVF International Conference on Computer Vision*, 2023. 3, 5
- [15] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. 3
- [16] Yuheng Li, Haotian Liu, Qingyang Wu, Fangzhou Mu, Jianwei Yang, Jianfeng Gao, Chunyuan Li, and Yong Jae Lee. GLIGEN: Open-set grounded text-to-image generation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. 3
- [17] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. In *Advances in Neural Information Processing Systems*, 2023. 1, 3
- [18] Shilong Liu, Zhaoyang Zeng, Tianhe Ren, Feng Li, Hao Zhang, Jie Yang, Chunyuan Li, Jianwei Yang, Hang Su, Jun Zhu, and Lei Zhang. Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection. In *European Conference on Computer Vision*, 2024. 3
- [19] Simeng Lu, Michele Mancini, Andrew Zisserman, and Mario Fritz. InfographicNLP: Expanding the scope of infographic analysis. *arXiv preprint arXiv:2310.13632*, 2023. 3
- [20] Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. ChartQA: A benchmark for question answering about charts with visual and logical reasoning. In *Findings of the Association for Computational Linguistics: ACL 2022*, 2022. 3
- [21] Minesh Mathew, Dimosthenis Karatzas, and C.V. Jawahar. DocVQA: A dataset for VQA on document images. In *IEEE/CVF Winter Conference on Applications of Computer Vision*, 2021. 3
- [22] Tamara Munzner. *Visualization Analysis and Design*. CRC Press, 2014. 3
- [23] NVIDIA. LocateAnything-3B: A visual grounding model. Hugging Face model card, <https://huggingface.co/nvidia/LocateAnything-3B>, 2025. Accessed 2026. 3, 5
- [24] Hoque Obeid and Enamul Hoque. Chart-to-Text: Generating textual descriptions of charts. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, 2020. 3
- [25] OpenAI. GPT-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 1
- [26] Dustin Podell, Zion English, Kyle Lacey, Andreas Blattmann, Tim Dockhorn, Jonas Müller, Joe Penna, and Robin Rombach. SDXL: Improving latent diffusion models for high-resolution image synthesis. In *International Conference on Learning Representations*, 2024. 1, 3
- [27] Aditya Ramesh, Prfulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with CLIP latents. *arXiv preprint arXiv:2204.06125*, 2022. 1, 3
- [28] Nikhila Ravi, Valentin Gabeur, Yuan-Ting Hu, Ronghang Hu, Chaitanya Ryali, Tengyu Ma, Haitham Khedr, Roman Rädle, Chloe Rolland, Laura Gustafson, et al. SAM 2: Segment anything in images and videos. *arXiv preprint arXiv:2408.00714*, 2024. 3, 5
- [29] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. 3
- [30] Chitwan Saharia, William Chan, Saurabh Saxena, Lala Li, Jay Whang, Emily Denton, Seyed Kamyar Ghasemipour, Burcu Karagol Ayan, S Sara Mahdavi, R Konuk Sarrafpour, et al. Photorealistic text-to-image diffusion models with deep language understanding. In *Advances in Neural Information Processing Systems*, 2022. 1, 3
- [31] Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, 2023. 3

²<https://github.com/wencanjiang/ChatImage>

- [32] Yifan Tang, Ao Zhang, Hengzhu Wang, Juan Li, and Zhijiang Hu. ChartLlama: A multimodal LLM for chart understanding and generation. *arXiv preprint arXiv:2311.16483*, 2023. [3](#)
- [33] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H Chi, Quoc V Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022. [3](#)
- [34] Xiaomi LLM-Core Team. MiMo-VL: Xiaomi MiMo vision-language model. Model release, <https://github.com/XiaomiMiMo/MiMo-VL>, 2025. Used in this work via the `mimo-v2.5` chat-completion vision endpoint. [3](#), [5](#)
- [35] Bin Yan, Yi Zhu, Jiangshan Wang, Zhihang Li, Yichao Zhang, Wu Liu, Erjin Zhang, and Jian Sun. UNINEXT: Universal instance perception as object-in-context prompting. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023. [3](#)
- [36] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. [3](#)
- [37] Lvmin Zhang, Anyi Rao, and Maneesh Agrawala. Adding conditional control to text-to-image diffusion models. In *IEEE/CVF International Conference on Computer Vision*, 2023. [3](#)
- [38] Yong Zhang. Mindmap: A creative visual thinking tool for education. *Journal of Educational Technology Systems*, 2014. [3](#)